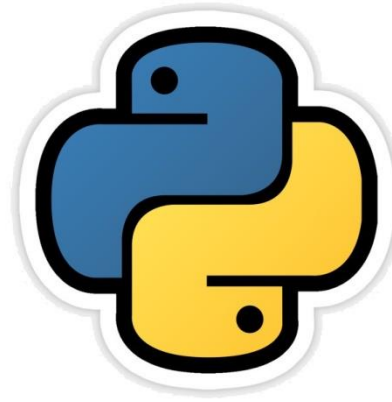


PYTHON BASICS की समीक्षा

सीबीएसई पाठ्यक्रम पर आधारित

कक्षा -12

अध्याय -1



द्वारा:

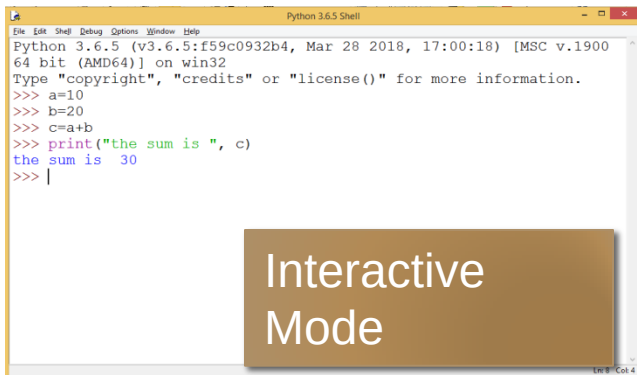
संजीव भदौरिया

स्नातकोत्तर शिक्षक (संगणक विज्ञान)

के० वि० बाराबंकी (लखनऊ संभाग)

Python (एक कंप्यूटर भाषा)

- पिछली कक्षा में हमने python के बारे में अच्छी तरह से अध्ययन किया | अब हम कक्षा 12 में पुनः इसका अवलोकन करेंगे और कुछ नई तकनीकों के बारे में सीखेंगे |
- पिछले पाठ्यक्रम में हमने यह जाना की Python एक पावरफुल और high level language है और यह interpreted भाषा है और इसे interpreter के द्वारा execute किया जाता है |
- Python हमें 2 प्रकार के मोड प्रदान करता है जिन पर हम कार्य कर सकते हैं -
 - Interactive mode (जहाँ हम सारे कमांड >>> पर देते हैं |)
 - Script mode (जहाँ हम program लिखते हैं |)



The screenshot shows the Python 3.6.5 Shell window. The title bar reads 'Python 3.6.5 Shell'. The menu bar includes 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The main text area displays the following content: 'Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 17:00:18) [MSC v.1900 64 bit (AMD64)] on win32', 'Type "copyright", "credits" or "license()" for more information.', and a series of commands and their outputs: '>>> a=10', '>>> b=20', '>>> c=a+b', and '>>> print("the sum is ", c)' followed by the output 'the sum is 30'. A brown box with the text 'Interactive Mode' is overlaid at the bottom right of the window.

```
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 17:00:18) [MSC v.1900
64 bit (AMD64)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> a=10
>>> b=20
>>> c=a+b
>>> print("the sum is ", c)
the sum is 30
>>> |
```

Interactive Mode



The screenshot shows a Python script file named 'MyFirstProgram.py' in a text editor. The title bar reads 'MyFirstProgram.py - C:/Users/KVBBKServer/AppData/Local/Programs/Python/Python36/MyFirstProgram.py (3.6.5)'. The menu bar includes 'File', 'Edit', 'Format', 'Run', 'Options', 'Window', and 'Help'. The main text area contains the following code: 'a=10', 'b=34', 'c=a+b', and 'print("The sum is ", c)'. A brown box with the text 'ScriptMode' is overlaid at the bottom right of the window.

```
a=10
b=34
c=a+b
print("The sum is ", c)
```

ScriptMode

Python (एक कंप्यूटर भाषा)

- विविध प्रकार की APPs बनाई जा सकती हैं जैसे –
 - GUI Apps
 - Web Apps
 - Games
 - DBMS Apps
 - Scripting etc.

Python (एक कंप्यूटर भाषा) कुछ कमियाँ

पाइथन में कुछ कमियाँ भी हैं जो की नज़रंदाज़ की जा सकती हैं -

- Fast Language नहीं हैं |
- लाइब्रेरीज कम हैं |
- Type binding में कमज़ोर है |
- आसानी से दूसरी भाषा में परिवर्तित नहीं की जा सकती है |

Tokens

- Token किसी भी कंप्यूटर भाषा की सबसे छोटी इकाई(smallest unit) होती है | इसे Lexical Unit भी कहते हैं | जो निम्न प्रकार के होते हैं -
 - Keywords
 - Identifiers (Names)
 - Literals
 - Operators
 - Punctuators

Keywords

- Keywords वे शब्द होते हैं जो कम्पाइलर के लिए एक विशेष अर्थ या सन्देश भेजते हैं |
- इन्हें विशेष कार्यों हेतु आरक्षित रखा गया है |
- इन्हें सामान्य नामों अथवा identifiers के तौर पर प्रयोग नहीं कर सकते हैं |
- पाइथन भाषा में निम्नलिखित keywords होते हैं |

False	class	finally	is	return
None	continue	for	lambda	try
True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

Identifiers

- ये किसी भी program के निर्माणकर्ता होते हैं तथा program के विभिन्न भागों को नामित करने के लिए इनका प्रयोग करते हैं, जैसे - variable, objects, classes, functions इत्यादि के नाम देने में ।
- एक identifier अक्षरों और अंको का सार्थक मिश्रण होता है जो किसी का नाम हो सकता है ।
- किसी भी identifier का पहला अक्षर कोई letter ही होगा या अंडरस्कोर (_) होगा ।
- बड़े अक्षर (upper case) तथा छोटे अक्षर (lower case) अलग अलग होते हैं ।
- 0 से लेकर 9 तक के सारे अंक इसका हिस्सा हो सकते हैं सिर्फ पहले अक्षर को छोड़कर ।
- Identifier की लम्बाई असीमित है तथा केस सेंसिटिव है ।
- किसी भी keyword को identifier के तौर पर नहीं लिया जा सकता ।
- किसी भी identifier में कोई भी विशेष सांकेतिक अक्षर की नहीं लिया जा सकता सिर्फ अंडरस्कोर (_) को ही लिया जा सकता है ।
- कुछ मान्य identifier –
 - Myfile, Date9_7_17, Z2T0Z9, _DS, _CHK FILE13 इत्यादि ।
- कुछ अमान्य identifier –
 - DATA-REC, 29COLOR, break, My.File इत्यादि ।

String Literals

- Literals को प्रायः निश्चित मान (Constant Values) कहा जाता है ।
- पाइथन कई प्रकार के literals को रखने के लिए अनुमति देता है -
 - String literals - "Pankaj"
 - Numeric literals – 10, 13.5, 3+5i
 - Boolean literals – True या False
 - Special Literal *None*
 - Literal collections

Literals / Values

- String Literal एक प्रकार का characters का एक क्रमिक समूह होता है जोकि कोड्स (" " या ' ') से घिरा रहता है ।
- पाइथन में string 2 प्रकार का होता है -
 - Single लाइन string
 - Text = "Hello World" या Text = 'Hello World'
 - Multi लाइन string
 - Text = 'hello \

या Text = ""hello
word ""

 world'

Numeric Literals

- Numeric values तीन प्रकार की हो सकती हैं -
 - int (signed integers)
 - Decimal Integer Literals – 10, 17, 210 इत्यादि |
 - Octal Integer Literals - 0o17, 0o217 इत्यादि |
 - Hexadecimal Integer Literals – 0x14, 0x2A4, 0xABD इत्यादि |
 - float (floating point real value)
 - Fractional Form – 2.0, 17.5 -13.5, -.00015 इत्यादि |
 - Exponent Form - -1.7E+8, .25E-4 इत्यादि |
 - complex (complex numbers)
 - 3+5i इत्यादि |

Boolean Literals

- ये 2 ही प्रकार के मान होते हैं True या False
 - A= True
 - B=False

Special Literals

- यह None होता जिसका मतलब होता है कुछ नहीं |
 - X = None

Operators

- Operator वह सांकेतिक अक्षर (Symbol) होते हैं जिनको किसी identifier पर प्रयोग करते हैं तो यह compute करके गणना करने में सक्षम होते हैं | ये identifier पर किसी न किसी कार्य को इंगित करते हैं |
- अतः किसी भी ऑपरेटर को कोई न कोई operand चाहिए ताकि वह उनके ऊपर गणना कर सके |

उदाहरण :

$$c = a + b$$

यहाँ a, b, c operand हैं तथा = और + operators हैं जिनका कार्य भिन्न भिन्न है |

Punctuators

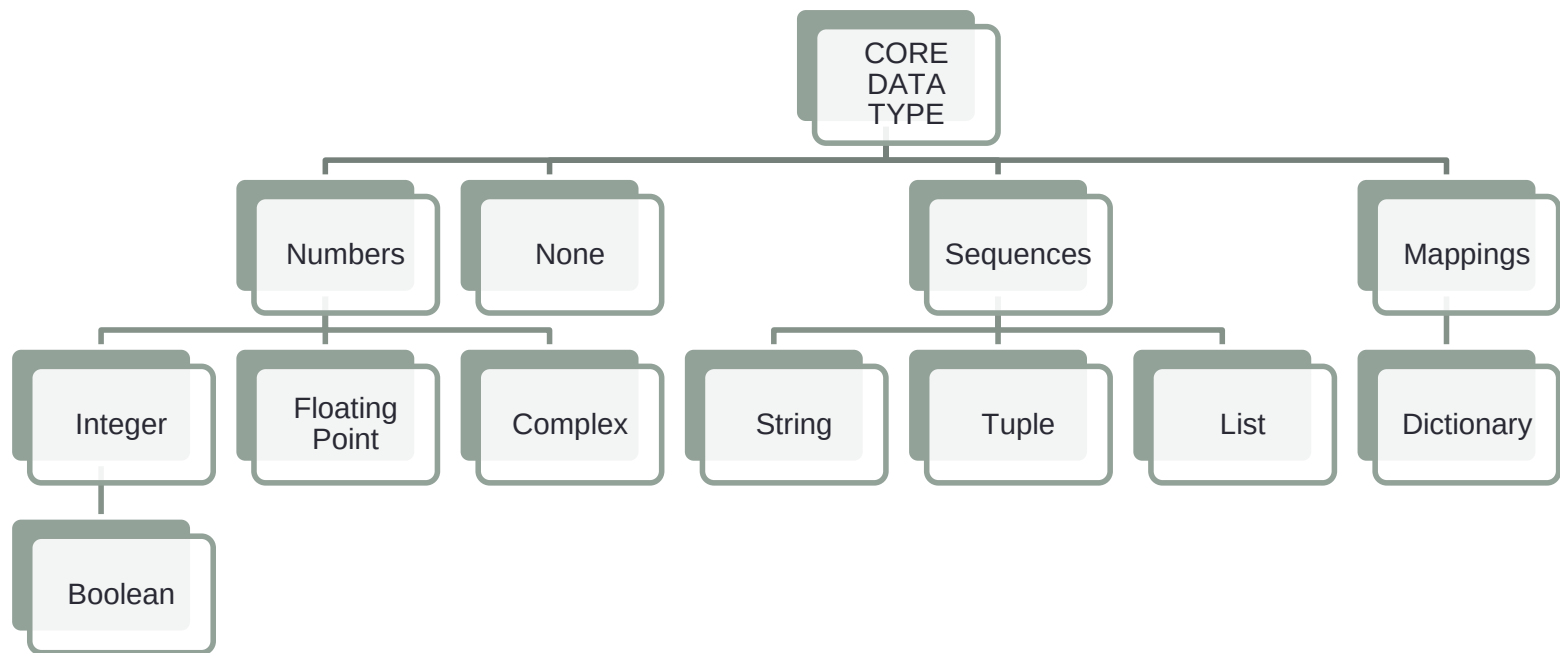
- पाइथन में punctuators का प्रयोग program के स्ट्रक्चर को बनाये रखने के लिए तथा निर्देशों (Instructions) एवं कथनों (Statements) के मध्य तालमेल बनाये रखने के लिए किया जाता है
- Python में Punctuators निम्न प्रकार के होते हैं -
' , ' , # , \ , (,) , [,] , { , } , @ , . , , , : , .. , ` , =

DATA TYPES

- डाटा किसी भी प्रकार का हो सकता है जैसे - character, integer, real, string इत्यादि ।
- पाइथन में “ ” के अंतर्गत लिखा गया कोई भी डाटा string डाटा होता है ।
- कोई भी सम्पूर्ण संख्या integer डाटा होती है ।
- कोई भी अपूर्ण संख्या अर्थात भिन्न या दशमलव (.) के साथ कोई भी संख्या real डाटा होती है ।
- True या False डाटा boolean को प्रदर्शित करती है
- पाइथन निम्न core data types का समर्थन करती है
 - Numbers (int जैसे 10, 5) (float जैसे 3.5, 302.24) (complex जैसे 3+5i)
 - String (जैसे “pankaj”, ‘pankaj’, ‘a’, “a”)
 - List जैसे [3,4,5,”pankaj”] इसके अवयव परिवर्तनीय (Mutable) हैं ।
 - Tuple जैसे (3,4,5,”pankaj”) इसके अवयव अपरिवर्तनीय (immutable) हैं ।
 - Dictionary जैसे {‘a’:1, ‘e’:2, ‘l’:3, ‘o’:4, ‘u’:5} जहाँ a,e,i,o,u key हैं और 1,2,3,4,5 क्रमशः उनकी values

CORE DATA TYPES

निम्न चित्र आपको पाइथन के Core Data Type को समझने में मदद करेगा ।



Variables और Values

यहाँ पर एक समझने वाली बहुत महत्वपूर्ण बात ये है जो अन्य भाषाओं में नहीं होती कि

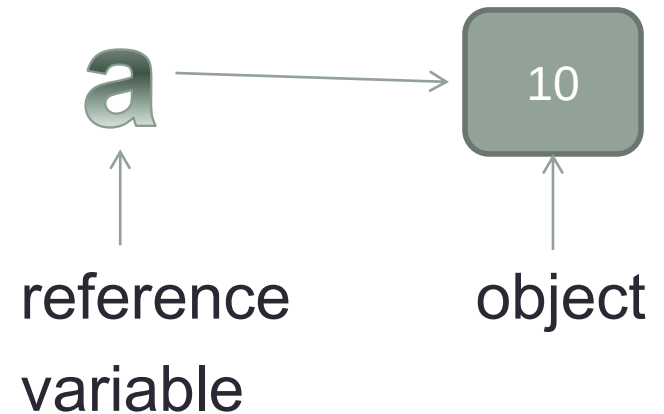
- पाइथन में values एक प्रकार के object होते हैं
- और variable name उन objects के reference name होते हैं |

इसको अब हम अच्छे से समझते हैं माना हम a variable में 10 assign करते हैं |

a = 10

यहाँ value 10 एक ऑब्जेक्ट है और a उस ऑब्जेक्ट का रिफरेन्स नाम है |

अब अगर इसी के आगे एक और variable b की वैल्यू 10 करें तो b भी इसी ऑब्जेक्ट को refer करेगा |



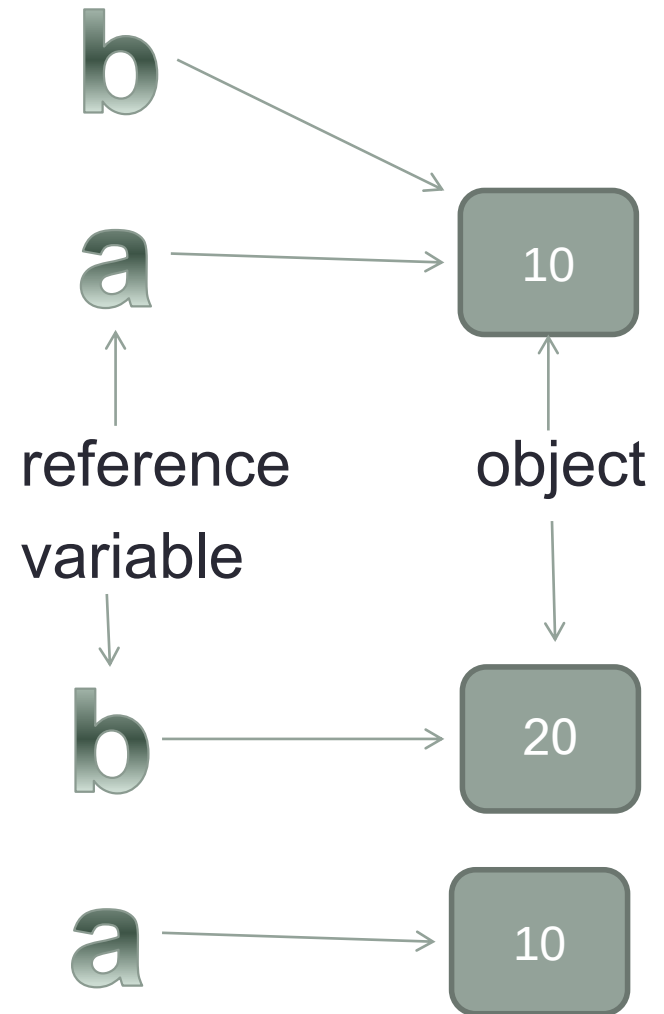
Variables और Values

अब अगर इसी के आगे एक और variable **b** की वैल्यू 10 करें तो **b** भी इसी ऑब्जेक्ट को refer करेगा | देखिये कैसे -

यहाँ हमारे पास variable 2 हैं लेकिन उनका लोकेशन एक ही है | अब यदि हम **b** की वैल्यू में बदलाव करेंगे अर्थात्

b=20

तब एक नया ऑब्जेक्ट बनेगा 20 नयी लोकेशन के साथ और इस ऑब्जेक्ट को refer करेगा **b**



Mutable और Immutable Types

- अतः पाइथन में डाटा objects को दो प्रकारों में वर्गीकृत किया जा सकता है
 - **Mutable (Changeable)**
 - इसके अंतर्गत lists, dictionaries और sets आते हैं
 - **Immutable (Non-Changeable)**
 - इसके अंतर्गत integers, floats, Booleans, strings और tuples आते हैं

Operators

- किसी भी operand (identifier) के ऊपर होने वाले operation (कार्य) अथवा specific task को दर्शाने वाला सांकेतिक अक्षर operator कहलाता है जैसे + , - , > , < इत्यादि ।
- पाइथन में निम्न प्रकार के operators होते हैं -
 - I. Arithmetic Operator
 - II. Relation Operator
 - III. Identity Operators
 - IV. Logical Operators
 - V. Bitwise Operators
 - VI. Membership Operators

Operator Associativity

- Python में किसी स्टेटमेंट में या expression में एक साथ कई operator लगे हों तो operator associativity हमेशा left-to-right रहेगी |

```
>>> 7*8/5//2
5.0
```

- इस उदाहरण में पहले $7*8$ होगा अर्थात 56, फिर इस 56 में 5 का भाग लगेगा अर्थात 11.2, तब 11.2 में 2 का पुनः // के द्वारा भाग लगेगा और उत्तर आयेगा |

सिर्फ ****** के सन्दर्भ में right-to-left associativity रहेगी |

```
>>> 3**3**2
19683
```

इस उदाहरण में $3**(3**2)$ के अनुसार गणना होगा |

Type Casting

- जैसा की हम जानते हैं कि यदि पाइथन के expression में datatypes mixed रूप में रहते हैं तो पाइथन कुछ operand का datatype आंतरिक रूप से बदल देता है | इस प्रक्रिया को implicit type conversion कहते हैं |
- एक और विकल्प हमारे पास explicit type conversion के रूप में हमारे पास होता है जिसके लिए हम निम्न प्रारूप अपनाते हैं –
- <datatype> (identifier)

जैसे – a="4"

b=int(a)

अथवा यदि हमारे पास a=5 और b=10.5 है तो हम a को float में बदल सकते हैं | d=float(a) करके |

- पाइथन में data conversion फंक्शन निम्न हैं

(1) int () (2) float() (3) complex() (4) str()
(5) bool()

Taking Input in Python

- Python में input लेने के लिए input () फंक्शन होता है जो की string के रूप में input लेता है फिर उसको आवश्यकता के अनुसार type cast करते हैं | जैसे किसी बेलन (cylinder) का आयतन (Volume) निकलने हेतु program निम्न है -

```
#Program to calculate vloume of a cone
radius=int(input("Enter the radius of the Cylinder : "))
height=int(input("Enter the height of the Cylinder : "))
volume = 3.14* radius*radius*height
print("The Volume of the cylinder is : ",volume)
|
```

- इसका output निम्नवत आएगा |

```
>>>
RESTART: C:/Users/KVBBKServer/AppData/Local/Programs/Python/Python36/VolOfCyl
inder.py
Enter the radius of the Cylinder : 10
Enter the height of the Cylinder : 5
The Volume of the cylinder is : 1570.0
>>> |
```

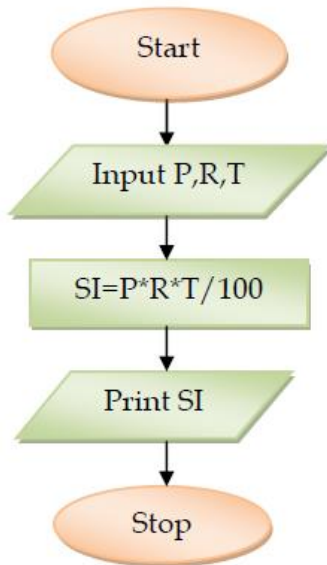
पाइथन में statements के प्रकार

- पाइथन में statement तीन प्रकार के होते हैं -
 - Empty Statements
 - pass
 - Simple Statements (Single Statement)
 - name=input ("Enter your Name ")
 - print(name) इत्यादि।
 - Compound Statements
 - **<Compound Statement Header>:**
 <Indented Body containing multiple simple statements/compound statements>
 - यहाँ Header लाइन keyword से शुरू होकर कोलन (:) पर समाप्त होती है।
 - एक body में एक से अधिक साधारण पाइथन statements या compound statements होते हैं।

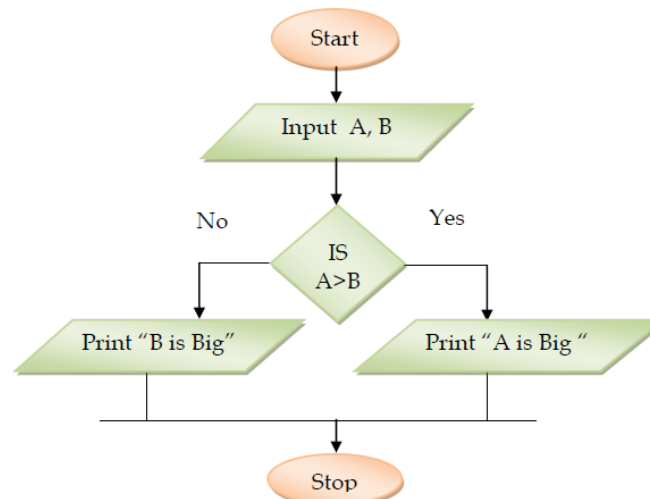
Statement Flow Control

- किसी भी program में statements या तो क्रम(sequencial) से क्रियान्वित(execute) होंगे या चयन करके(Selective) या फिर पुनरावृत्ति(Iteration) में क्रियान्वित होंगे ।

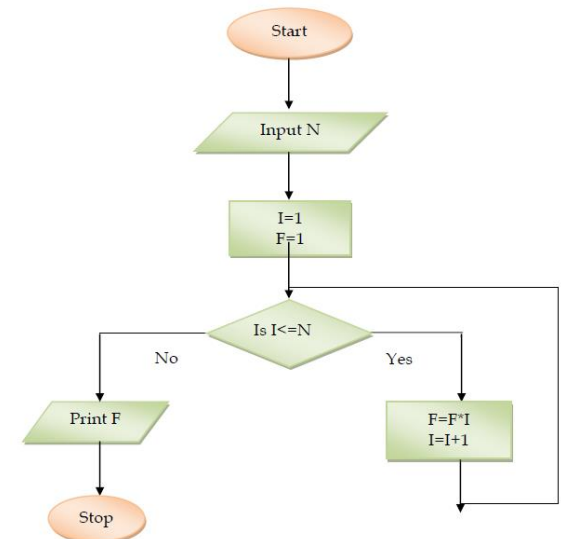
Sequential



Selective



Iterative

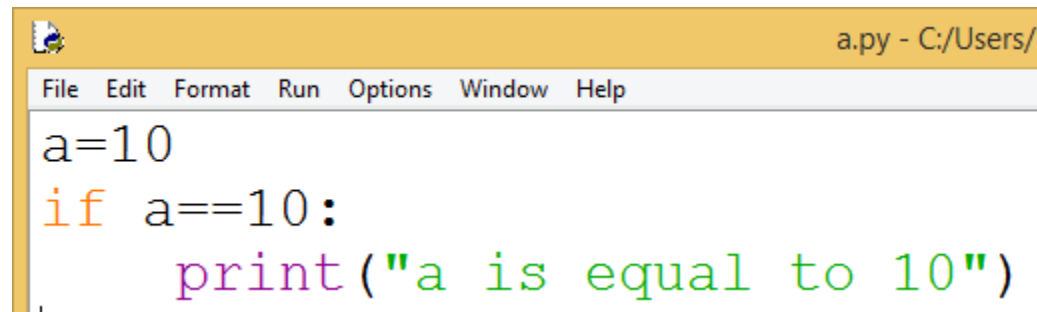


Python में if Statements

- Python में if statement का प्रयोग चयनित प्रोसेसिंग के लिए करते हैं | यदि किन्हीं दो statements में से कोई एक, किसी शर्त पर चयन करना हो तो if statement का प्रयोग किया जाता है | इसका syntax (प्रारूप) निम्नवत है -

```
if <condition>:  
    statement(s)
```

जैसे -



The screenshot shows a Python IDE window titled 'a.py - C:/Users/'. The menu bar includes File, Edit, Format, Run, Options, Window, and Help. The code editor contains the following Python code:

```
a=10  
if a==10:  
    print("a is equal to 10")
```

Python में if-else Statements

- Python में if-else statement का प्रयोग चयनित प्रोसेसिंग के लिए करते हैं | यदि किन्हीं दो statements में से कोई एक, किसी शर्त पर चयन करना हो तो if-else statement का प्रयोग किया जाता है | इसका syntax (प्रारूप) निम्नवत है -

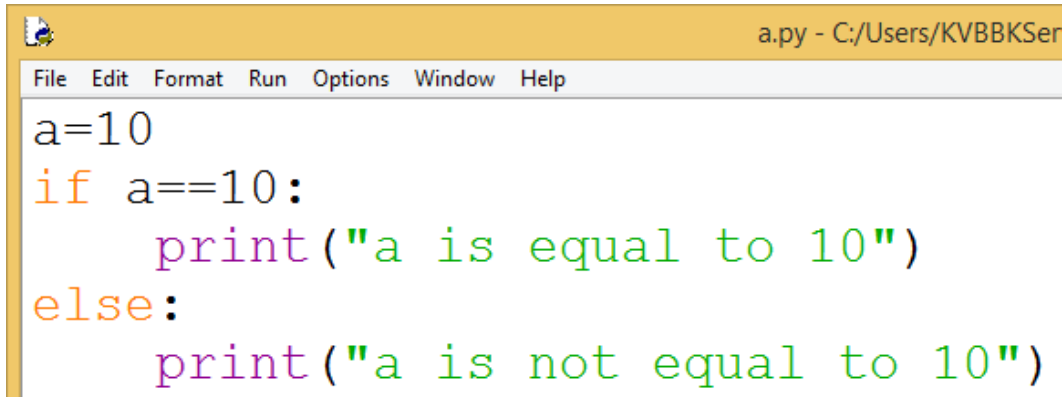
if <condition>:

statement(s) when condition is true

else:

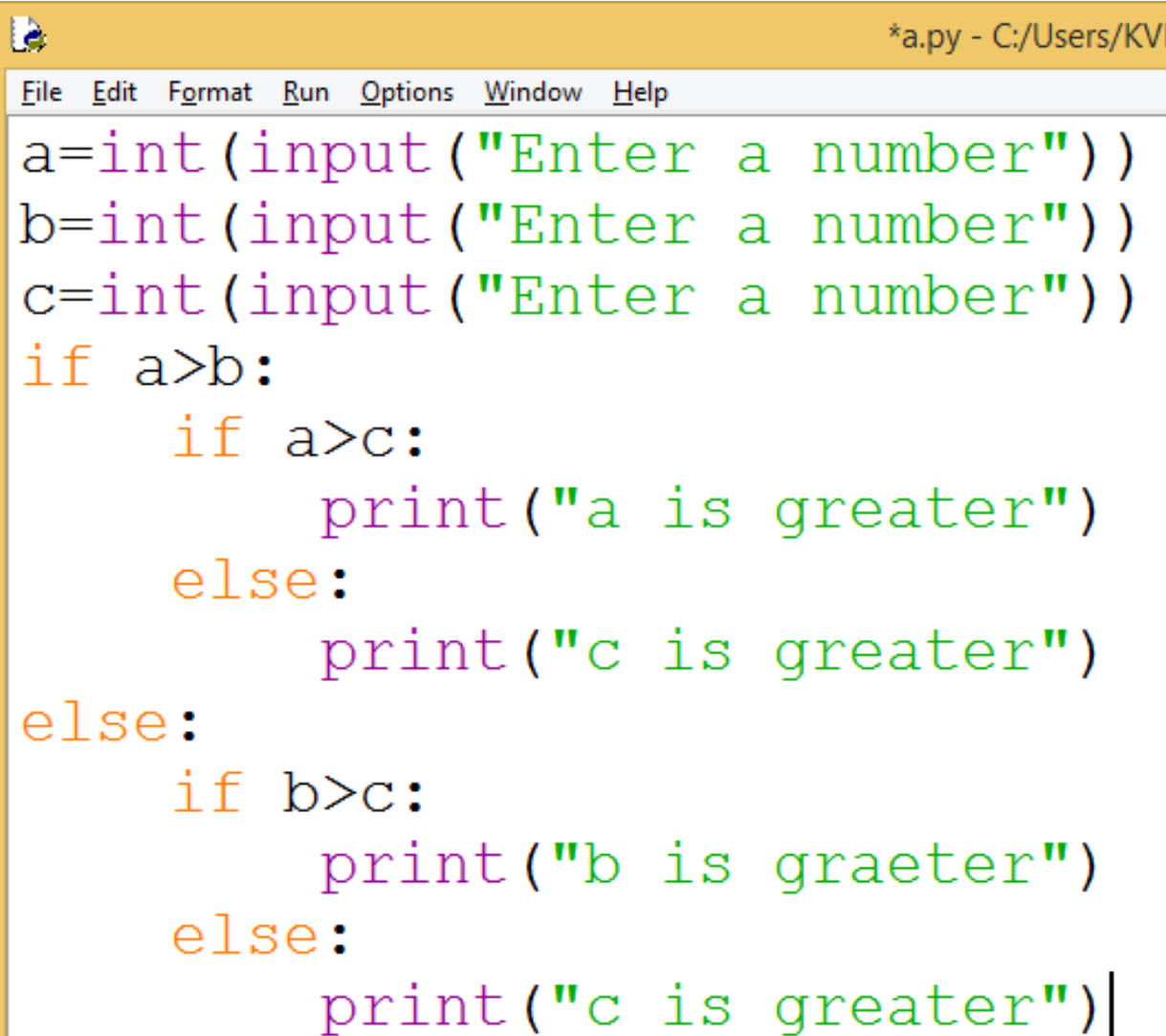
statement(s) when condition is false

जैसे -

A screenshot of a Python IDE window titled 'a.py - C:/Users/KVBBKSer'. The window has a menu bar with 'File', 'Edit', 'Format', 'Run', 'Options', 'Window', and 'Help'. The code inside the window is as follows:

```
a=10
if a==10:
    print("a is equal to 10")
else:
    print("a is not equal to 10")
```

Nested If -else



```
*a.py - C:/Users/KV
File Edit Format Run Options Window Help
a=int(input("Enter a number"))
b=int(input("Enter a number"))
c=int(input("Enter a number"))
if a>b:
    if a>c:
        print("a is greater")
    else:
        print("c is greater")
else:
    if b>c:
        print("b is graeter")
    else:
        print("c is greater")|
```

Loop/Repetitive Task/Iteration

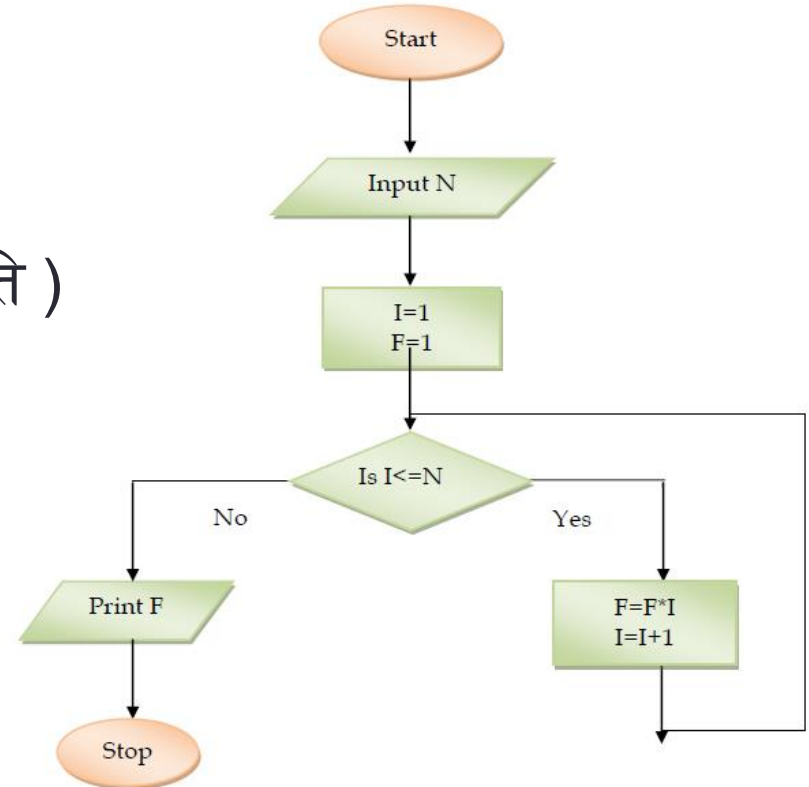
इस प्रकार के कार्यों में कुछ statements की तब तक पुनरावृत्ति होती है जब तक की सही स्थिति न आजाये ।

Loop में मुख्यतः 3 अवयव आवश्यक है -

1. Start (प्रारम्भिक मान देना)
2. Step (loop को आगे बढ़ने वाला)
3. Stop (loop को समाप्त करने की स्थिति)

पाइथन में loop निम्न हैं

- for loop (Counting based)
- while loop (Conditional Based)



range () Function

- **Python** में range() फंक्शन बहुत काम का है | इसके कार्य को समझने के लिए इसका syntax समझते हैं

range (<lower limit>,<upper limit>)

यदि हम लिखें - range (0,5)

तो यह एक list बनाता है [0,1,2,3,4] अर्थात यह list में lower limit से लेकर upper limit से 1 कम तक की सारे values को लेलेगा |

range (0,10,2) तो यह लिस्ट में [0,2,4,6,8] रखेगा |

range (5,0,-1) तो यह लिस्ट में [5,4,3,2,1] रखेगा |

range (0,10,2) तो यह लिस्ट में [0,2,4,6,8] रखेगा |

Jump Statements

break Statement

```
n=int(input("Enter a number"))
c=1
while c<11:
    if c==5:
        break
    print(n, "x", c, "=", c*n)
    c=c+1
```

Output

```
Enter a number4
4 x 1 = 4
4 x 2 = 8
4 x 3 = 12
4 x 4 = 16
```

```
n=int(input("Enter a number"))
c=1
for c in range(1,11):
    if c==5:
        break
    print(n, "x", c, "=", c*n)
|
```

Output

```
Enter a number5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
>>>
```

in तथा not in ऑपरेटर

- Python में *in* operator का निम्न कार्य है -
3 in [1,2,3,4] में ये statement **True** return करेगा |
5 in [1,2,3,4] में ये statement **False** return करेगा |
- Python में *not in* operator का निम्न कार्य है -
5 not in [1,2,3,4] में ये statement **True** return करेगा

```
for a in [1, 2, 3]:  
    print(a)  
    print(a*a)
```

Jump Statements

continue Statement

```
n=int(input("Enter a number"))
for c in range(1,11):
    if c==5:
        continue
    print(n, "x", c, "=", c*n)
```

```
n=int(input("Enter a number"))
c=0
while c<11:
    c=c+1
    if c==5:
        continue
    print(n, "x", c, "=", c*n)
```

दोनों program का output

```
Enter a number5
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
5 x 11 = 55
>>> |
```

Nested Loop

```
n=int(input("Enter the number"))  
for r in range(1,n+1):  
    for c in range(1,r+1):  
        print("*", end="")  
    print("")
```

OUTPUT

```
Enter the number5  
*  
**  
***  
****  
*****
```

String को बनाना

- String को हम निम्न प्रकार बना सकते हैं -

1. Direct variable को value assign करके

```
>>> str="I love my india"
```

String
Literal

```
>>> str
```

```
'I love my india'
```

2. Input लेकर

```
>>> str1=input("Enter a string")
```

```
Enter a stringThis is python
```

```
>>> str1
```

```
'This is python'
```

Input () सदैव दिए गए input को string के रूप में ही return करता है ।

एक string को travers करना

- किसी भी string के प्रत्येक character को प्राप्त करके उसको प्रयोग करना अथवा उसे प्रदर्शित करना (Display) करना traversing कहलाता है | जैसे

```
name="superb"
```

```
for ch in name:
    print(ch, "-", end="")
```

Output

s -u -p -e -r -b -

String को उल्टा करके print करने का एक program -

```
str=input("Enter a String")
print("The string ", str, " in reverse order is: ")
length=len(str)
for a in range(-1, (-length-1), -1):
    print(str[a], end="")
```

Output

```
Enter a Stringsanjeev
The string  sanjeev  in reverse order is:
veejnas
```

String Operators

- String पर 2 operators काम करते हैं + और * | string के केस में इनका काम अलग तरीके से होता है |

- **+** (इसका प्रयोग दो string को आपस में जोड़ने के लिए लिया जाता है)

- जैसे - "tea" + "pot" हो जायेगा "teapot"
- जैसे - "1" + "2" हो जायेगा "12"
- जैसे - "123" + "abc" हो जायेगा "123abc"

- ***** (इसका प्रयोग string के कई प्रतिरूप बनाने के लिए करते हैं |)

- जैसे - 5*"@" हो जायेगा "@@@"
- जैसे - "go!" * 3 हो जायेगा "go!go!go!"

नोट : - "5" * "6" ये expression अमान्य है |



String के टुकड़े करना (Slicing)

- निम्न उदाहरण पर ध्यान केन्द्रित करते हैं -

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13
Word	R	E	S	P	O	N	S	I	B	I	L	I	T	Y
Reverse index	-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

word = "RESPONSIBILITY"

word[0 : 14] का उत्तर आयेगा 'RESPONSIBILITY'

word[0 : 3] का उत्तर आयेगा 'RES'

word[2 : 5] का उत्तर आयेगा 'SPO'

word[-7 : -3] का उत्तर आयेगा 'IBIL'

word[: 14] का उत्तर आयेगा 'RESPONSIBILITY'

word[: 5] का उत्तर आयेगा 'RESPO'

word[3 :] का उत्तर आयेगा 'PONSIBILITY'

String के Functions

<u>String.capitalize()</u>	Converts first character to Capital Letter
<u>String.find()</u>	Returns the Lowest Index of Substring
<u>String.index()</u>	Returns Index of Substring
<u>String.isalnum()</u>	Checks Alphanumeric Character
<u>String.isalpha()</u>	Checks if All Characters are Alphabets
<u>String.isdigit()</u>	Checks Digit Characters
<u>String.islower()</u>	Checks if all Alphabets in a String are Lowercase
<u>String.isupper()</u>	returns if all characters are uppercase characters
<u>String.join()</u>	Returns a Concatenated String
<u>String.lower()</u>	returns lowercased string
<u>String.upper()</u>	returns uppercased string
<u>len()</u>	Returns Length of an Object
<u>ord()</u>	returns Unicode code point for Unicode character
<u>reversed()</u>	returns reversed iterator of a sequence
<u>slice()</u>	creates a slice object specified by range()

List को बनाना

- List पाइथन का एक मानक (standard) data type है | यह एक ऐसा sequence है जो किसी भी प्रकार के डाटा की सूची store कर सकता है |
- List को पाइथन में व्यक्त करने के लिए square brackets “ [] “ का प्रयोग करते हैं | उदहारण के तौर पर -
 - [] यह रिक्त list है (Empty list)
 - [1, 2, 3] यह integers की list है
 - [1, 2.5, 5.6, 9] यह numbers की list है (integer और float)
 - ['a', 'b', 'c'] यह characters की list है |
 - ['a', 1, 'b', 3.5, 'zero'] यह mixed values की list है |
 - ['one', 'two', 'three'] यह string की list है
- एक बार फिर से यह दोहरा लेते हैं कि python में list और dictionary mutable और बाकी समस्त data type immutable होते हैं |

List को बनाना

- List को बनाने के लिए निम्न तरीके हैं -

- Empty list बनाने के लिए -

`L = []`

- list बनाने के लिए आप निम्न statement का भी रयोग कर सकते हैं -

`L = list()`

```
>>> Mylist = list('hello')
>>> Mylist
['h', 'e', 'l', 'l', 'o']
>>>
```

```
>>> L = ('p', 'a', 'n', 'k', 'j')
>>> NewList = list(L)
>>> NewList
['p', 'a', 'n', 'k', 'j']
>>>
```

- Long lists बनाने के लिए -

`even = [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20]`

- Nested list बनाने के लिए -

`L = [3, 4, [5, 6], 7]`

एक तरीका यह भी हो सकता है

```
>>> L1 = list(input("Enter List Elements"))
Enter List Elements12345
>>> L1
['1', '2', '3', '4', '5']
```

ये tuple है

List को बनाना

- जैसा की हमने इस उदहारण में देखा कि list में भले ही हमने नंबर में मान दिया हो परन्तु list में मान string के रूप में ही गया है

```
>>> L1 = list(input("Enter List Elements"))
Enter List Elements12345
>>> L1
['1', '2', '3', '4', '5']
```

- यदि हमें numeric रूप में ही value को input करना है तो list के साथ निम्न function लगाना पड़ेगा -

eval(input())

L=eval(input("Enter list to be added "))

eval () function किसी string को पास करने पर उसके type को identify करके return करता है ।

```
>>> a="15"
>>> b="25"
>>> print(a+b)
1525
>>> print(eval(a)+eval(b))
40
```

String Values

एक अन्य उदहारण देखें

```
>>> a="15"
>>> b=eval(a)
>>> type(a)
<class 'str'>
>>> type(b)
<class 'int'>
>>>
```

List को Access करना

- List को access करना सीखने से पहले हम list और string में समानता देख लें।
- List एक प्रकार से string के जैसी ही sequence होती है।
- List भी अपने प्रत्येक अवयव (Element) का index बनाती है।
- String के जैसे इसमें भी 2 index होती है एक forward index (0, 1, 2, 3,n-1 तक) और एक backward index (-n से -1 तक)।

Forward index	0	1	2	3	4	5	6	7	8	9	10	11	12	13
List	R	E	S	P	O	N	S	I	B	I	L	I	T	Y
Backward index	-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

- इसमें भी आप string के जैसे ही values को access कर सकते हैं।

```
>>> vowels=['a','e','i','o','u']
>>> vowels[4]
'u'
>>> vowels[-5]
'a'
>>> vowels[-1]
'u'
```

List को Access करना

- List की लम्बाई (Length) को पता करने के लिए `len()` function का प्रयोग करते हैं |


```
>>> name=list("Pankaj")
>>> name
['P', 'a', 'n', 'k', 'a', 'j']
>>> len(name)
6
>>>
```
- `L[i]` यह `i` index पर जो भी वैल्यू है उसको return करता है
- `L[i:j]` एक नयी list return करता है जिसमे `L` के `i` index और `j` index के मध्य की समस्त values होती हैं इसमें `j` index की वैल्यू शामिल है |

```
>>> name=list("Pankaj")
>>> name[3]
'k'
>>> nm=name[2:4]
>>> nm
['n', 'k']
```

Important 1: List के साथ membership operator (*in*, *not in*) दोनों कार्य ठीक वैसे ही करते हैं जैसे की अन्य sequences में।

Important 2: + operator किसी list को अन्य list के आखिरी में जोड़ती है जबकि * किसी भी list को repeat करता है

List और String में अंतर

- List और string में बस यही एक basic difference है की string immutable है और list mutable।
- String की individual values को बदला नहीं जा सकता जबकि list में ऐसा

```
>>> string="aeiou"
>>> string[2]
'i'
>>> string[2]='I'
Traceback (most recent call last):
  File "<pyshell#2>", line 1, in <module>
    string[2]='I'
TypeError: 'str' object does not support item assignment
```

String में value नहीं बदली | error आगई |

List में value बदल गयी | मतलब साफ़ है की list mutable होती है |

```
>>> st=list("aeiou")
>>> st
['a', 'e', 'i', 'o', 'u']
>>> st[2]='I'
>>> st
['a', 'e', 'I', 'o', 'u']
>>>
```

List को travers करना

- List को travers करने का तात्पर्य उसके प्रत्येक अवयव(element) को access करना और उनको process करना है।
- List का traversal को अत्यंत आसान बनाता है for loop –

```
for <item> in <list>:
```

```
# प्रत्येक item को यहाँ process करें
```

```
L= ['P', 'Y', 'T', 'H', 'O', 'N']
for a in L:
    print(a)
```

```
L= ['P', 'Y', 'T', 'H', 'O', 'N']
```

```
length=len(L)
```

```
for a in range(length):
```

```
    print("Index ",a," और ",(a-length)," पर आइटम है : ",L[a])
```

```
RESTART: C:/
```

P
Y
T
H
O
N

```
RESTART: C:/Users/KVBBKServer/
```

```
Index 0 और -6 पर आइटम है : P
Index 1 और -5 पर आइटम है : Y
Index 2 और -4 पर आइटम है : T
Index 3 और -3 पर आइटम है : H
Index 4 और -2 पर आइटम है : O
Index 5 और -1 पर आइटम है : N
```

Python UNICODE को समर्थन करता है अतः हिंदी में भी output संभव है।

List Operations (+, *)

- List पर होने वाले प्रमुख operation हैं joining list, replicating list और list की slicing(टुकड़े) |
- List को ज्वाइन करने के लिए ***+ operator*** का प्रयोग किया जाता है जो पहली list के आखिरी में दूसरी list को जोड़ देता है | + operator के साथ दोनों operands को list type का होना चाहिए, अन्यथा error आयेगी |

```
>>> L1=[1, 2, 3]
>>> L2=[4, 5, 6, 7]
>>> L3=L1+L2
>>> L3
[1, 2, 3, 4, 5, 6, 7]
```

- List को replicate करने के लिए **** operator*** का प्रयोग किया जाता है |

```
>>> L1=[1, 2, 3]
>>> L2=L1*3
>>> L2
[1, 2, 3, 1, 2, 3, 1, 2, 3]
```

List Operations (Slicing)

- List को slice करने के लिए syntax है - `seq = list [ start : stop ]`

```
>>> LST=[10,12,14,20,22,24,30,32,34]
```

```
>>> SEQ=LST[3:-3]
```

```
>>> SEQ
```

```
[20, 22, 24]
```

```
>>> SEQ=LST[2:4]
```

```
>>> SEQ
```

```
[14, 20]
```

- List को slice करने के लिए एक और syntax है -

`seq=list[start:stop:step]`

```
>>> LST=[10,12,14,20,22,24,30,32,34]
```

```
>>> SEQ=LST[0:10:2]
```

```
>>> SEQ
```

```
[10, 14, 22, 30, 34]
```

```
>>> LST[2:10:3]
```

```
[14, 24, 34]
```

```
>>> LST[::3]
```

```
[10, 20, 30]
```

```
>>> LST[::-1]
```

```
[34, 32, 30, 24, 22, 20, 14, 12, 10]
```

Slice का प्रयोग list Modification के लिए

- निम्नलिखित उदाहरणों पर ध्यान केन्द्रित करिए ।

```
>>> L=["one","two","three"]
```

```
>>> L
```

```
['one', 'two', 'three']
```

```
>>> L[0:2]=[0,1] ←————
```

यहाँ नयी values assign की जा रही हैं।

```
>>> L
```

```
[0, 1, 'three']
```

```
>>> L=["one","two","three"]
```

```
>>> L[0:2]="a" ←————
```

यहाँ भी नयी values assign की जा रही हैं।

```
>>> L
```

```
['a', 'three'] ←————
```

दोनों के परिणामों में अंतर को परखिये ।

```
>>> l=[1,2,3]
```

```
>>> l[2:]="604"
```

```
>>> l
```

```
[1, 2, '6', '0', '4']
```

```
>>> l[2:]=144 ←————
```

144 एक संख्या है न की

```
Traceback (most recent call last): sequence
```

```
File "<pyshell#12>", line 1, in <module>
```

```
l[2:]=144
```

```
TypeError: can only assign an iterable
```

List Functions और Methods

- List manipulation के लिए पाइथन कुछ built-in फंक्शन भी प्रदान करता है | list के फंक्शन को कॉल करने का प्रारूप निम्न है - `<list-object>.<method-name>`

Function	Details
List.index(<item>)	यह पास किये गए आइटम का index return करता है
List.append(<item>)	यह पास किये item को list के आखिरी में जोड़ता है
List.extend(<list>)	यह एक list को argument के रूप में लेके सम्बंधित list(जिसके साथ फंक्शन कॉल किया गया है) के आखिरी में पूरी list जुड़ जाती है
List.insert(<pos>,<item>)	यह फंक्शन दिए गए स्थान (pos) पर दिया गया item insert कर देता है
List.pop(<index>)	यह दिए गए index वाले item को delete करके return करता है इसमें index को पास करना वैकल्पिक होता है यदि इसे पास न करें तो लास्ट item pop होगा
List.remove(<value>)	यह केवल दिए गए value को delete करेगा वह भी जो list में पहले मिलेगा, लेकिन delete की गयी value को return नहीं करेगा

List Functions और Methods

Function	Details
List.clear ()	यह list के सारी values को remove कर देता है और empty list बना देता है
List.count (<item>)	यह दिए गए item को count करके return करता है कि list में पास किया गया item कितने बार आया है
List.reverse ()	यह फंक्शन list को reverse आर्डर में set कर देता है यह कोई नयी list नहीं बनता है
List.sort ()	यह फंक्शन list को बढ़ते क्रम से set कर देता है यदि हमें घटते क्रम में list को set करना है तो list.sort(reverse =True) लिखना होगा

Tuple को बनाना

- Python में tuple को बनाने के लिए “()” कोष्ठक का प्रयोग किया जाता है ।

()	empty tuple
(1, 2, 3)	integers का tuple
(1, 2.5, 3.7, 7)	numbers का tuple
('a', 'b', 'c')	characters का tuple
('a', 1, 'b', 3.5, 'zero')	mixed values का tuple
('one', 'two', 'three', 'four')	string का tuple

Tuple एक प्रकार के immutable sequence होते हैं जिनके किसी भी मान में हम परिवर्तन नहीं कर सकते।

Tuple को बनाना

tuple बनाने के लिए निम्न उदाहरण पर ध्यान दें -

- Empty tuple:



```
>>> t=()
>>> t
()
```

- Single element tuple:



```
>>> t=(1)
>>> t
1
```

- Long tuple:

```
>>> t=(0,1,4,9,16,25,36,49,64,81,100,121)
>>> t
(0, 1, 4, 9, 16, 25, 36, 49, 64, 81, 100, 121)
```

- Nested tuple:



```
>>> t=(1,2,3,(4,5))
>>> t
(1, 2, 3, (4, 5))
```

Tuple को बनाना

tuple को अन्य sequences से बनाने के लिए निम्न उदाहरण पर ध्यान दें इसके लिए tuple() फंक्शन का प्रयोग करते हैं।-

String से tuple बनाना

```
>>> t=tuple("Hello")
>>> t
('H', 'e', 'l', 'l', 'o')
```

List से tuple बनाना

```
>>> L=['a', 'e', 'i', 'o', 'u']
>>> T=tuple(L)
>>> T
('a', 'e', 'i', 'o', 'u')
```

Input लेकर tuple बनाना

```
>>> t1=tuple(input("Enter element"))
Enter element123456
>>> t1
('1', '2', '3', '4', '5', '6')
```

ये सभी character type के element बन गए हैं। यदि इन्हें विभिन्न प्रकार के elements बनाना है तो निम्न statement लिखना होगा।

```
Tuple=eval(input("Enter elements"))
```

```
>>> t1=eval(input("Enter the elements"))
Enter the elements (2,4.5,"hello")
>>> t1
(2, 4.5, 'hello')
```


Tuple को Access करना

- Python में tuple को access करने के लिए आप वे सभी कार्य कर सकते हैं जो की हमने list के साथ किये थे | list के जैसे ही हम tuple के प्रत्येक element को access कर सकते हैं |
- List के साथ समानता:** list के जैसे tuple में भी index होता है | tuple और tuple में mutability को छोड़कर सारे गुण समान हैं |

Forward index	0	1	2	3	4	5	6	7	8	9	10	11	12	13
Tuple	R	E	S	P	O	N	S	I	B	I	L	I	T	Y
Backward index	-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

- List की लम्बाई पता करने के लिए len () फंक्शन का प्रयोग करते हैं

```
>>> t=tuple("Hello")
>>> len(t)
5
```

Tuple को Access करना

- Indexing और Slicing:

- $T[i]$ index i पर उपस्थित item को return करता है |
- $T[i:j]$ ये एक नया tuple return करता है जिसमे T के i से j index तक के item होते हैं |
- $T[i:j:n]$ ये एक नया tuple return करता है जिसमे T के i से j index तक के n वें (n^{th}) item होते हैं |

```
>>> T=(1,2,3,4,5,6,7,8,9)
>>> T[1:10:3]
(2, 5, 8)
```

- Membership operator:

- Tuple में membership operator “in” तथा “not in” ठीक वैसे ही कार्य करते हैं जैसे की list में करते हैं| (कृपया इसके list वाले अध्याय को देखें)

- Concatenation और Replication operators:

- Tuple के साथ + operator दूसरे tuple को पहले tuple के बाद में जोड़ देता है| जबकि * tuple के elements को repeat कर देता है|

Tuple को Access करना

- Individual elements को access करना (उदहारण देखें)-

```
>>> L=['a', 'e', 'i', 'o', 'u']
>>> L[0]
'a'
>>> L[3]
'o'
```

- Tuple को travers करने के लिए निम्न code लिखना होगा –

```
for <item> in <tuple>:
    #प्रत्येक item को process करिए
```

```
T=tuple("Python")
print(T, end="")
print()
for a in T:
    print(a)
```

```
('P', 'y', 't', 'h', 'o', 'n')
P
y
t
h
o
n
OUTPUT
T
```

Tuple Operations

- Tuple को join करना
 - जोड़ने के लिए + के साथ दोनों tuple होने चाहिए

```
>>> tp1=(1,3,5)
>>> tp2=(6,7,8)
>>> tp1+tp2
(1, 3, 5, 6, 7, 8)
>>> tp3=tp1+tp2
>>> tp3
(1, 3, 5, 6, 7, 8)
```

- Tuple को join करने में निम्न में error आयेगी

- Tuple + number करने में
- Tuple + complex number करने में
- Tuple + string करने में
- Tuple + list करने में
- Tuple + (5) करने में भी error आयेगी क्योंकि single वैल्यू के साथ tuple को वैल्यू ही माना जाता है न की tuple |

```
>>> tp1=(1,3,5)
>>> tp1*3
(1, 3, 5, 1, 3, 5, 1, 3, 5)
>>> tp1
(1, 3, 5)
>>> tp2=tp1*3
>>> tp2
(1, 3, 5, 1, 3, 5, 1, 3, 5)
```

- Tuple को Replicate करना

Tuple की Slicing

```
>>> tp1=(10,12,14,20,22,24,30,32,34)
```

```
>>> seq=tp1[3:-3]
```

```
>>> seq
```

```
(20, 22, 24)
```

Upper limit चाहे जो हो tuple पूरी list के अंतिम item तक ही दिखायेगा

```
>>> tp1[3:30]
```

```
(20, 22, 24, 30, 32, 34)
```

```
>>> tp1[-15:7]
```

```
(10, 12, 14, 20, 22, 24, 30)
```

```
>>> tp1[0:10:2]
```

हर दूसरा item दिखायेगा |

```
(10, 14, 22, 30, 34)
```

```
>>> tp1[0:10:3]
```

हर तीसरा item दिखायेगा |

```
(10, 20, 30)
```

```
>>> tp1[::3]
```

```
(10, 20, 30)
```

Dictionary बनाना

- Dictionary बनाने के लिए आपको “ { } ” के अंतर्गत key:value के pairs को संग्रहीत करना होता है ।

- `<dictionary-name>={ <key1>:<value1>,<key2>:<value2>,<key3>:<value3> . . . }`

उदाहरण:

`teachers={"Rajeev":"Math", "APA":"Physics", "APS":"Chemistry", "SB":"CS"}`

उपरोक्त उदाहरण में :

Key-value pair	Key	Value
<code>"Rajeev":"Math"</code>	<code>"Rajeev"</code>	<code>"Math"</code>
<code>"APA":"Physics"</code>	<code>"APA"</code>	<code>"Physics"</code>
<code>"APS":"Chemistry"</code>	<code>"APS"</code>	<code>"Chemistry"</code>
<code>"SB":"CS"</code>	<code>"SB"</code>	<code>"CS"</code>

Dictionary बनाना

- Dictionary के कुछ सामान्य उदाहरण निम्न हैं –

Dict1= { } # यह एक empty dictionary है जिसमे कोई भी element नहीं है ।

DayofMonth= { "January":31, "February":28, "March":31, "April":30, "May":31, "June":30,
"July":31, "August":31, "September":30, "October":31, "November":30,
"December":31 }

FurnitureCount = { "Table":10, "Chair":13, "Desk":16, "Stool":15, "Rack":15 }

- उपरोक्त उदाहरणों से आप समझ सकते हैं कि keys और उनकी सम्बंधित values कौन कौन सी हैं ।
- बस यहाँ एक ध्यान रखने वाली बात ये है कि keys सदैव immutable type की रहनी चाहिए ।

Note: Dictionary को associative array या mapping या hashes भी कहते हैं ।

Dictionary बनाना

- बस यहाँ एक ध्यान रखने वाली बात ये है कि keys सदैव immutable type की रहनी चाहिए ।
- यदि आप mutable type की key बनाने की कोशिश करते हैं तो पाइथन error दिखायेगा । उदहारण के लिए -

```
>>> dict = {[2,3]: "MyRoom"}  
Traceback (most recent call last):  
  File "<pyshell#0>", line 1, in <module>  
    dict = {[2,3]: "MyRoom"}  
TypeError: unhashable type: 'list'
```

यहाँ key एक list है जो कि mutable type की है

यहाँ इस error का मतलब है कि अपने एक ऐसी key ली है जिसका type mutable है और पाइथन इसको समर्थन नहीं करता है ।

Dictionary को Access करना

- जब भी हमें dictionary से value access करना होता है तो हम key का प्रयोग करते हैं, ठीक वैसे ही जैसे list से वैल्यू को access करने के लिए index का प्रयोग करते हैं।
- Key: value के जोड़े से हमें key का पता चलता है।

```
teachers={"Rajeev":"Math", "APA":"Physics", "APS":"Chemistry", "SB":"CS"}
```

- उपरोक्त उदाहरण से यदि हम निम्न statement को क्रियान्वित करें तो -

```
>>> teachers={"Rajeev":"Math", "APA":"Physics", "APS":"Chemistry", "SB":"CS"}
>>> teachers["Rajeev"]
'Math'
>>> print("Rajeev Teaches ", teachers["Rajeev"])
Rajeev Teaches  Math
```

- यदि हमने key "Rajeev" को चुना और print किया तो उसकी वैल्यू math आयी। एक अन्य उदाहरण देखें तो -

```
>>> d={"Vowel1":"'a'", "Vowel2":"'e'", "Vowel3":"'i'", "Vowel4":"'o'", "Vowel5":"'u'"}
>>> print(d["Vowel2"])
e
>>> print(d["Vowel5"])
u
```

यदि आप किसी नॉन key को access के लिए देते हैं तो error आजायेगी।

Dictionary को Traverse करना

- Dictionary को traverse करने के लिए for loop का प्रयोग करते हैं जिसका प्रारूप निम्न है -

```
for <item> in <dictionary>:
    प्रत्येक item को यहाँ प्रोसेस करें |
```

```
>>> d={5:"number", "a":"String", (1,2):"tuple"}
>>> for k in d:
        print(k, " : ", d[k])
```

```
5      :   number
a      :   String
(1, 2) :   tuple
```

यहाँ गौर करने वाली बात ये है कि dictionary *d* के प्रत्येक जोड़े की key, loop के ***k variable*** में आ रही है | उसके बाद print statement के साथ दिए गए फॉर्मेट में हम output ले सकते हैं |

Assignment : आप अपने मित्रों की एक फ़ोन dictionary बनाइये जिसमे key, मित्र का नाम हो और वैल्यू, उसका फ़ोन नंबर हो |

Dictionary को Traverse करना

- यदि हमें key और values को access करना हो तो keys() और values() फंक्शन का प्रयोग करते हैं | इस हेतु निम्न statement लिखते हैं -

```
>>> d={"Vowel1":'a', "Vowel2":'e', "Vowel3":'i', "Vowel4":'o', "Vowel5":'u'}
>>> d.keys()
dict_keys(['Vowel1', 'Vowel2', 'Vowel3', 'Vowel4', 'Vowel5'])
>>> d.values()
dict_values(['a', 'e', 'i', 'o', 'u'])
```

- d.keys() फंक्शन सिर्फ key को प्रदर्शित करेगा |
- d.values () फंक्शन सिर्फ values को प्रदर्शित करेगा |

Dictionary के गुण

1. **Unordered set:** dictionary एक प्रकार का अव्यवस्थित key:value जोड़े का समूह होता है
2. **ये sequence नहीं होता है:** list, string और tuple की तरह यह sequence नहीं होता है क्योंकि यह एक प्रकार का elements का अव्यवस्थित समूह होता है | जबकि sequence को संख्याओं से क्रम में indexed किया जाता है जिसके कारण वे क्रमित (Ordered) होती हैं |
3. इनकी indexing करने के लिए keys का प्रयोग किया जाता है तथा पाइथन के अनुसार key कोई भी immutable type की हो सकती है | चूँकि string और number immutable होते हैं अतः आप इन्हें key के रूप में ले सकते हैं| विभिन्न keys के साथ एक उदाहरण निम्न है -

```
>>> d={0:"Key0",1:"Key1","3":"KeyAsString",(4,5):"KeyAsTuple","Hello":6}
>>> d[0]
'Key0'
>>> d[1]
'Key1'
>>> d["3"]
'KeyAsString'
>>> d[(4,5)]
'KeyAsTuple'
>>> d["Hello"]
6
```

Dictionary की keys हमेशा immutable type की होनी चाहिए, जैसे number, string या tuple. जबकि dictionary की value किसी भी प्रकार की हो सकती है |

Dictionary के गुण

4. **Keys** को **unique** होना चाहिए: चूँकि **keys** का प्रयोग **values** को identify करने के लिए किया जाता है अतः **keys** का **unique** होना आवश्यक है ।
5. जबकि दो **unique keys** की **values** एक सामान हो सकती हैं ।
6. Dictionary परिवर्तनीय (**mutable**) है इसमें हम किसी निश्चित **key** की **value** को बदल सकते हैं । इसके लिए निम्न **syntax** का प्रयोग कर सकते हैं ।

```
<dictionary>[<key>] = <value>
```

```
>>> d={0:"Key0",1:"Key1","3":"KeyAsString", (4,5):"KeyAsTuple","Hello":6}
>>> d["3"]="This is String"
>>> d
{0: 'Key0', 1: 'Key1', '3': 'This is String', (4, 5): 'KeyAsTuple', 'Hello': 6}
```

7. आन्तरिक रूप से यह **mapping** के रूप में संग्रहीत(store) होती है। इसके **key:value** जोड़े एक दूसरे से एक आंतरिक फंक्शन के द्वारा सम्बंधित रहते हैं ये फंक्शन **hash-function**** कहलाते हैं । link करने की इस विधा को **mapping** कहते हैं।

****Hash-function**, key और value को map और link करने की एक आंतरिक(internal) algorithm होती है ।

Dictionary के साथ काम करना

- यहाँ पर हम dictionary पर होने वाले विभिन्न operation के बारे में चर्चा करेंगे, जैसे dictionary में element को जोड़ना (adding), सुधार करना (Update), delete करना इत्यादि | लेकिन उसके पहले dictionary बनाना सीखते हैं |

- dictionary को initialize करना:** इसके लिये हम key:value जोड़े को कोमा (,) से पृथक (separate) करते हुए उनका एक समूह बनाते हैं और उस समूह को मझले कोष्ठक “ { } ” में रख देते हैं | जैसे -

```
>>> Employee={'name':'suresh','salary':15000,'age':34}
>>> Employee
{'name': 'suresh', 'salary': 15000, 'age': 34}
```

- Empty dictionary में key:value जोड़े को add करना:** empty dictionary बनाने के लिए दो तरीके हैं इनमे से कोई एक तरीका अपनाये -

- Employee = { }
- Employee = dict()

उसके बाद उसमे निम्न syntax लगायें

<dictionary>[<key>] = <value>

```
>>> Employee = {}
>>> Employee['name']='Pankaj'
>>> Employee['salary']=20000
>>> Employee
{'name': 'Pankaj', 'salary': 20000}
```

Dictionary के साथ काम करना

3. Dictionary को name और value जोड़े के साथ बनाना : इसके अंतर्गत dict() constructor का प्रयोग करके key और value के जोड़े बनाकर dictionary बनाई जा सकती है | ऐसा करने के कई तरीके हैं

I. Key:value pair को argument के रूप में पास करके :

```
>>> Employee=dict(name='Ramesh',salary=10000,age=24)
>>> Employee
{'name': 'Ramesh', 'salary': 10000, 'age': 24}
```

इसमें गौर करिए की dictionary में name, salary एवं age पर अपने आप single inverted comma आगया है जबकि argument में नहीं लगाया गया है।

```
>>> Employee=dict({'name':'Rahul','age':24})
>>> Employee
{'name': 'Rahul', 'age': 24}
```

Dictionary के साथ काम करना

III. Keys को अलग और values को अलग specify करके:

इसके लिए dict () constructor के अन्दर zip () फंक्शन का प्रयोग करेंगे -

```
>>> Employee=dict(zip(('name','salary','age'),('Mukesh',12000,26)))
>>> Employee
{'name': 'Mukesh', 'salary': 12000, 'age': 26}
```

```
>>> Employee=dict([['name','anand'], ['salary',14000], ['age',23]])
>>> Employee
{'name': 'anand', 'salary': 14000, 'age': 23}
```

List पास करके

```
>>> Employee=dict([('name','Angad'), ('salary',11000), ('age',29)])
>>> Employee
{'name': 'Angad', 'salary': 11000, 'age': 29}
```

List की tuple पास करके

```
>>> Employee=dict((('name','Suman'), ('salary',17000), ('age',21)))
>>> Employee
{'name': 'Suman', 'salary': 17000, 'age': 21}
```

Tuple की tuple पास करके

Dictionary में element को जोड़ना

Dictionary में element को जोड़ने के लिए निम्न syntax का प्रयोग करते हैं ।

`<dictionary>[<key>]=<value>`

```
>>> Employee=dict((( 'name', 'Suman'), ('salary',17000), ('age',21)))
>>> Employee
{'name': 'Suman', 'salary': 17000, 'age': 21}
>>> Employee['Dept']='Sales'
>>> Employee
{'name': 'Suman', 'salary': 17000, 'age': 21, 'Dept': 'Sales'}
```

Dictionary में Nesting

निम्न उदाहरण पर ध्यान दें जिसमें dictionary के elements में dictionary ही है ।

```
Employee={'mukesh':{'age':23,'salary':34000},'Meena':{'age':27,'salary':24000}}
for key in Employee:
    print("Employee",key,':')
    print('Age: ',str(Employee[key]['age']))
    print('Salary: ',str(Employee[key]['salary']))
```

```
Employee mukesh :
Age: 23
Salary: 34000
Employee Meena :
Age: 27
Salary: 24000
```

Dictionary में element को सुधारना (Updation)

Dictionary में element को सुधारने के लिए निम्न syntax का प्रयोग करते हैं ।

`<dictionary>[<ExistingKey>]=<value>`

```
>>> Employee={'name':'Sudha','Salary':10000,'age':24}
>>> Employee['Salary']=15000
>>> Employee
{'name': 'Sudha', 'Salary': 15000, 'age': 24}
```

WAP to create a dictionary containing names of employee as key and their salary as value.

Output

```
File Edit Format Run Options Window Help
n=int(input("Enter the Number of Employees"))
Employee={} #Empty Dictionary
for a in range(n):
    key=input("Enter Name of the Employee")
    value=int(input("Enter Salary of Employee"))
    Employee[key]=value
print("The Dictionary is now is :")
print(Employee)
```

```
Enter the Number of Employees3
Enter Name of the EmployeePavan
Enter Salary of Employee30000
Enter Name of the EmployeeRakesh
Enter Salary of Employee12000
Enter Name of the EmployeeMukesh
Enter Salary of Employee20000
The Dictionary is now is :
{'Pavan': 30000, 'Rakesh': 12000, 'Mukesh': 20000}
```

Dictionary से element को Delete करना

Dictionary से element को delete करने के लिए निम्न दो syntaxes का प्रयोग करते हैं | delete करने के लिए key का उपस्थित होना ज़रूरी है अन्यथा पाइथन error दे देगा|

1. `del <dictionary>[<key>]` ये सीधे delete करता है बिना deleted value को return नहीं करता है |

```
>>> emp={'Pavan': 30000, 'Rakesh': 12000, 'Mukesh': 20000}
>>> del emp['Pavan']
>>> emp
{'Rakesh': 12000, 'Mukesh': 20000}
```

Delete करने के बाद value return नहीं हुई

2. `<dictionary>.pop(<key>)` ये element delete करने के साथ साथ deleted value को return भी करता है |

```
>>> emp={'Pavan': 30000, 'Rakesh': 12000, 'Mukesh': 20000}
>>> emp.pop('Rakesh')
12000
>>> emp.pop('Aman', "Not Found")
'Not Found'
```

Delete करने के बाद value return हुई

ऐसा करने से यदि key नहीं match होती है तो साथ में दिया गया सन्देश print होजाता है |

Dictionary में element की उपस्थिति पता करना

Dictionary में element की उपस्थिति पता करने के लिए membership operator का प्रयोग करते हैं |

1. `<key> in <dictionary>` यह key उपस्थित होने पर True देता है अन्यथा False
2. `<key> not in <dictionary>` यह key के न उपस्थित होने पर True देता है अन्यथा False

```
>>> emp={'Pavan': 30000, 'Rakesh': 12000, 'Mukesh': 20000}
>>> 'Pavan' in emp
True
>>> 'Hari' in emp
False
>>> 'Hari' not in emp
True
>>> 'Pavan' not in emp
False
```

ध्यान रखने वाली यह बात है की *in* और *not in* values पर लागू नहीं होते। ये सिर्फ key के साथ ही कार्य करेंगे ।

Dictionary की Pretty Printing

Dictionary को सही से सजा के print करने के लिए *json module* को import करना होगा | उसके बाद dumps () के निम्न syntax का प्रयोग करना होगा |

```
json.dumps(<>,indent=<n>)
```

```
>>> import json
>>> emp={'Pavan': 30000, 'Rakesh': 12000, 'Mukesh': 20000}
>>> print(json.dumps(emp,indent=2))
{
    "Pavan": 30000,
    "Rakesh": 12000,
    "Mukesh": 20000
}
```

एक program जो किसी वाक्य में शब्दों की गणना करके dictionary बनाएगा

```
import json
statement="His Name is Pankaj. \
His father is a teacher. His \
father is a good person"
w=statement.split()
d={}
for c in w:
    key=c
    if key not in d:
        count=w.count(key)
        d[key]=count
print("Counting frequencies in list\n",w)
print(json.dumps(d,indent=1))
```

```
Counting frequencies in list
['His', 'Name', 'is', 'Pankaj.', '
His', 'father', 'is', 'a', 'teacher
.', 'His', 'father', 'is', 'a', 'go
od', 'person']
{
  "His": 3,
  "Name": 1,
  "is": 3,
  "Pankaj.": 1,
  "father": 2,
  "a": 2,
  "teacher.": 1,
  "good": 1,
  "person": 1
}
```

यहाँ शब्द और उनकी बारंबारता (frequency) की एक dictionary बन गयी है ।

Dictionary Function और Method

1. ***len()*** Method : यह dictionary की लम्बाई बताता है ।

```
>>> emp={'Pavan': 30000, 'Rakesh': 12000, 'Mukesh': 20000}
>>> len(emp)
3
```

2. ***clear()*** Method : यह dictionary को खाली कर empty बना देता है ।

```
>>> emp={'Pavan': 30000, 'Rakesh': 12000, 'Mukesh': 20000}
>>> emp.clear()
>>> emp
{}
```

3. ***get()*** Method : यह दिए गए key की value को return करता है ।

```
>>> emp={'Pavan': 30000, 'Rakesh': 12000, 'Mukesh': 20000}
>>> emp.get('Rakesh')
12000
>>> emp.get('Ankit', "Not Found")
'Not Found'
```

यह <dictionary>[<key>] की भांति ही कार्य करता है।

इसमें यदि key न मिले तो default सन्देश भी दिया जा सकता है ।

Dictionary Function और Method

4. **items()** Method : यह dictionary के समस्त items को (key:value) के tuple के रूप में return करता है ।

```
>>> emp={'Pavan': 30000, 'Rakesh': 12000, 'Mukesh': 20000}
>>> mylist=emp.items()
>>> mylist
dict_items([('Pavan', 30000), ('Rakesh', 12000), ('Mukesh', 20000)])
```

5. **keys()** Method : यह dictionary के keys की list return करता है ।

```
>>> emp={'Pavan': 30000, 'Rakesh': 12000, 'Mukesh': 20000}
>>> emp.keys()
dict_keys(['Pavan', 'Rakesh', 'Mukesh'])
```

6. **values()** Method : यह dictionary के values की list return करता है

```
>>> emp={'Pavan': 30000, 'Rakesh': 12000, 'Mukesh': 20000}
>>> emp.values()
dict_values([30000, 12000, 20000])
```


Dictionary Function और Method

7. **Update () Method**: यह फंक्शन किसी dictionary में दूसरी dictionary के key:value जोड़े का विलय (merge) कर देता है | इसमें आवश्यकतानुसार परिवर्तन तथा addition दोनों संभव हैं | उदहारण :

```
>>> emp1={'name':'Suresh','salary':10000,'age':24}
>>> emp2={'name':'siya','salary':45000,'dept':'sales'}
>>> emp1.update(emp2)
>>> emp1
{'name': 'siya', 'salary': 45000, 'age': 24, 'dept': 'sales'}
>>> emp2
{'name': 'siya', 'salary': 45000, 'dept': 'sales'}
```

उपरोक्त उदहारण में यदि आप देखें तो emp1 में सामान key के values में परिवर्तन हुआ जबकि असमान key अपनी value के साथ जुड़ गयी |

- अब हम अगली सेशन में पाइथन भाषा के बेसिक सीखेंगे |
- कृपया हमारे ब्लॉग पर हमें अनुसरित करें | हमारा ब्लॉग है -

www.pythontrends.wordpress.com